

Instituto Tecnológico Sudamericano

Carrera:

Desarrollo de Software

Tema:

Tarea en Clase , Prompt usados

Realizado por:

Julián Zúñiga

Docente:

Jostin Vacacela

Materia:

Herramientas Informáticas Para El Despliegue De
Diagramas

Fecha de entrega :

10 de Octubre del 2026



Registro del Proceso de Consulta y Desarrollo

1. Resumen Ejecutivo del Proceso

El objetivo principal de la interacción fue la implementación de un recurso User (CRUD básico) en NestJS, comenzando desde la corrección de comandos de generación CLI y resolviendo problemas de integración de datos en memoria, estructura de clases/DTOs y pruebas de endpoints mediante Postman.

2. Mapeo de Prompts, Objetivos y Palabras Clave

Consulta 1: Creación inicial del recurso y corrección de sintaxis

Prompt principal / Contexto: nest new generate resources user

Objetivo: Intentar crear y configurar el módulo/servicio/controlador del usuario.

Palabras clave: NestJS, nest generate resource, PrismaService, CRUD, CommonJS vs ESM.

Resultado u Orientación: La IA identificó la confusión en el comando de la CLI (mezclar nest new con nest generate) y entregó el código base de un recurso funcional.

Consulta 2: Centralización de metadatos en un archivo auxiliar

Prompt principal / Contexto: ya cree el modulo de usuario ahora necesito guardar los end points en un array en un archivo que cree llamado temporal.ts

Objetivo: Guardar en un arreglo la lista de endpoints para documentación o testing.

Palabras clave: temporal.ts, userEndpoints, export const, rutas HTTP.

Resultado u Orientación: Se estructuró un arreglo de cadenas de texto con las rutas (POST /users, GET /users, etc.) exportables desde temporal.ts.

Consulta 3: Definición e integración de la clase/entidad User

Prompt principal / Contexto: necesito crear una clase para el usuario y subsecuentemente pero dentro del archivo que guarda el array



Objetivo: Unificar la definición de la clase User dentro del mismo archivo temporal.ts junto con el arreglo de endpoints.

Palabras clave: class User, constructor, Partial<User>, Object.assign, entidad.

Resultado u Orientación: Se agregaron la entidad User y el array de endpoints en un único archivo exportable.

Consulta 4: Refactorización del constructor y resolución de errores de persistencia

Prompt principal / Contexto: arregla este constructor y ya lo cree pero no me aparece nada en el temporal.ts

Objetivo: Corregir el constructor de User para instanciar propiedades directamente y entender por qué temporal.ts no se actualiza dinámicamente con las peticiones POST.

Palabras clave: constructor(id, name, email), persistencia en memoria, private users: User[], UserService.

Resultado u Orientación: Se aclaró que temporal.ts es un archivo de definiciones estáticas (TypeScript) y que el almacenamiento de usuarios generados en tiempo de ejecución debe gestionarse en un arreglo users dentro de UserService.

Consulta 5: Implementación de DTOs y pruebas en Postman

Prompt principal / Contexto: las respuestas solo me dice la acción, mi create user está vacío y como lo pruebo con postman

Objetivo: Reemplazar las cadenas de texto simuladas que generó la CLI por lógica funcional con un DTO que valide el cuerpo de la petición.

Palabras clave: CreateUserDto, updateUserDto, class-validator, @IsString(), @IsEmail(), Postman, POST http://localhost:3000/user.

Resultado u Orientación: Se redactaron los DTOs para validar entradas (name, email) y se conectó el servicio para guardar los datos recibidos mediante peticiones HTTP.

3. Estado Final del Desarrollo

Entidad y Endpoints estáticos: Definidos en temporal.ts.



Transferencia de Datos (DTOs): CreateUserDto y UpdateUserDto configurados con decoradores de class-validator.

Logica de Negocio (UserService): Métodos create, findAll, findOne, update y remove conectados a una variable privada de clase users: User[] como almacenamiento temporal en memoria.

Verificación: Lista para ser testeada enviando payloads JSON desde Postman.

